Parallel GPU Hash Tables: Thread Cooperation, Memory Access, and Contention

Jiaxi Zhang jiaxizha@andrew.cmu.edu
Hongzheng Li hongzhel@andrew.cmu.edu

Project Proposal

URL

https://suqichen777.github.io/15418-Project-Webpage/

Summary

We will implement and optimize a concurrent hash table in CUDA that supports parallel insertion, look-up, and potentially deletion or supporting dynamic size. We will explore how collision handling, atomic operations, and GPU memory access patterns affect performance. We will evaluate throughput against CPU and baseline GPU implementations across different table sizes, load factors and key distributions.

Background

Hash tables make significant contribution to key-value storage, database indexing, duplicate detection, and other data-processing workloads. Considering the independence of different key, it seems that large batches of insertions or lookups can be handled with parallelism, while hash collisions and shared access to entries introduce dependency against the expectation.

Thus, we would like to focus on the potential concurrency improvement for a fixed-capacity hash table in GPU memory in our project to leverage the issue discussed above. Our initial implementation will use integer keys and values, and reserved empty slots to form a large hash table on GPU. We intend to accelerate large batches of insertion and lookup to exploit the GPU's parallelism capability and high memory bindwidth.

We intend to use open addressing with linear probing as the baseline: a initial slot is found by the hash function in one operation, and move forward successively to examine the following slots, until it finds the key, or the next available slot. In the baseline, it will assign one GPU thread to each operation. To prevent concurrent insertions from overwriting each other, we plan to use atomic CAS operation to replace an empty slot with the complete record.

Thread Cooperation: Then we will investigate the potential cooperation among a group of GPU threads, building upon the baseline, facing large batches of operations: multiple threads can examine one candidate's slot at the same time. As a result, we will research further into the tradeoff between processing more independent operations simultaneously, and allocating more GPU threads for a single operation. Duplicate insertions will make no change to the table, receiving successful results, while inserting into a full table with a non-existing key will get a failure. We will also explore ways to reduce atomic contention and unnecessary CAS attempts through multi-thread cooperative probing and coordination. Pseudocode for cooperative insertion with a thread group (All threads in a group are expected to execute the code cooperatively):

Algorithm 1 Cooperative Insertion with a Thread Group

Require: Table T with capacity M; key-value pair (k, v); group size G
Ensure: Inserted, AlreadyPresent, or Full

```
1: function CooperativeInsert(T, k, v)
2:     ℓ ← thread's rank within the group
3:     h ← hash(k) mod M
4:     o ← 0
5:     while o < M do
6:         a ← (o + ℓ < M)
7:         p ← (h + o + ℓ) mod M
8:         r ← AtomicLoad(T[p]) if a
9:         if GroupAny(a ∧ r.key = k) then
10:            return AlreadyPresent
11:        end if
12:        j ← FirstEmptyLane(a, r)
13:        if j = None then
14:            o ← o + G
15:        else
16:            if ℓ = j then
17:                old ← AtomicCAS(T[p], Empty, (k, v))
18:            end if
19:            old ← Broadcast(old, j)
20:            if old = Empty then
21:                return Inserted
22:            else if old.key = k then
23:                return AlreadyPresent
24:            end if                          ▷ On contention, recheck the same window
25:        end if
26:    end while
27:    return Full
28: end function
```

Our implementation will proceed through the following stages:

- Parallel insertion followed by parallel lookup
- Concurrent insertion and lookup
- Potential deletion or Dynamic resizing

It should be noted that our core implementation will still use a fixed-capacity table. Deletion and resizing will be considered as a stretch goal after the core part has been evaluated.

The Challenge

Workload and Constraints

1. Concurrent insertions can target the same hash table slots, which requires synchronization and creating contention. Atomic CAS operations can prevent confliction, but reduces throughput at the same time.

2. Hashing produces irregular memory accesses that limit memory coalescing, and collisions cause threads to follow probing sequences of different lengths, leading to warp divergence and uneven work distribution
3. As the number of threads increases, the overhead of coordinating those threads would increase. Finding the sweet spot where the overhead does not outweigh the benefits of additional parallelism is important

Learning Objectives

1. Learn how to balance the trade-offs between maximizing parallelism while also minimizing synchronization overhead
2. Learn the difference between implementing hash table in CPU and in GPU
3. Utilizing tolls to distinguish bottlenecks caused by different constraints: contention, divergence, memory access, and so on.

Resources

- **Machines:** A multicore CPU host and CUDA-capable NVIDIA GPU on GHC. We will record both the CPU model, the GPU model, and other metadata for later experiment comparison.
- **Starting code base:** We will implement the hash table, the described baseline and cooperative algorithm, and benchmark harness from scratch. We intend to use CUDA atomic operations and cooperative-group primitives as the basic synchronization methods, and might change as needed. We will take NNIDIA cuCollections [2] as an external comparison.
- **References:** NVIDIA's GPU hash-map article provides background on open addressing and cooperative probing [1]; The cuCollection repository provides a practical comparison [2].
- **Workloads / datasets:** We plan to generate batches of integer keys and values which should be reproducible so that we can vary table size, load factor, operation hit rate and duplicate frequency, or potential stress test. We do not require external datasets.
- **Still needed:** N/A (temporarily)

Goals and Deliverables

Plan to Achieve

1. Implement a fixed-capacity GPU hash table using C++, CUDA, open addressing and linear probing.
2. The hash table shall support insertion and lookup
3. Establish a set of benchmarks to measure the performance of the hash table, and under the same configuration (same load factor, same key distribution, etc.), achieve 60% of the performance as NVIDIA cuCollections [2]

Hope to Achieve

1. Support dynamic size with an efficient resizing mechanism, using lazy resizing as a mechanism to balance the latency caused by resizing
2. Achieve 80% of the performance as NVIDIA cuCollections [2] under the same configuration (same load factor, same key distribution, etc.)
3. Explore separate chaining and compare it with linear probing to understand why NVIDIA cuCollections [2] choose the latter over the former.

Fallback (if work goes more slowly)

1. Achieve 50% of the performance as NVIDIA cuCollections [2] under the same configuration (same load factor, same key distribution, etc.)

Demo

We plan to record a video to showcase that, under different kinds of key distribution, the performance of

Platform Choice

We plan to choose C++/CUDA because they provide direct control over low-levek thread organization, synchronization, and other operations. We can also control the memory layout with their support.

We plan to use large batches of integer keys and values because they can provide enough parallel workload, taking advantage of GPU threads. It help construct a subtle comparison between CPU hash tables and GPU hash tables as well. The hash collisions and irregular memory access implied by the dataset also provide the measurement criteria in our project.

Schedule

Week	Dates	Tasks	Owner
1	Nov 9 – Nov 15	Submit proposal (Mon Nov 9). Confirm hardware access, record metadata and confirm interfaces. Build a CPU baseline reference, GPU initial insertion and lookup, and workload generator.	Jiaxi: hardware reference & baseline reference. Hongzheng: CUDA setup & workload generator.
2	Nov 16 – Nov 22	Complete baseline tests, implement the first cooperative probing mechanism, collect initial experiment data, establish cuCollection comparisons.	Jiaxi: tests and comparison harness. Hongzheng: cooperative implementation. Both: experiment profiling.
3	Nov 23 – Nov 29	**Milestone report due Mon Nov 23.** Begin mixed-operation support and targeted concurrency tests. Debugging and profiling is the main focus.	Jiaxi: mixed-operation semantics and tests. Hongzheng: group-size and memory-access experiments
4	Nov 30 – Dec 6	Complete main experiment with mixed-operation validation, collect experiment data and measure performance. Start extended goals if the core goals are complete.	Both.
5	Dec 7 – Dec 11	**Final report due Thu Dec 10; poster session Fri Dec 11.** Consolidate the final results. Complete final report and prepare the poster presentation.	Both.

References

[1] Daniel Juenger, Nico Iskos, Yunsong Wang, Jake Hemstad, Christian Hundt, and Nikolay Sakharnykh. Maximizing performance with massively parallel hash maps on GPUs. NVIDIA Technical Blog, March 2023. `https://developer.nvidia.com/blog/maximizing-performance-with-massively-parallel-hash-maps-on-gpus/`. Accessed: 2026-11-09.

[2] NVIDIA. cuCollections: Gpu-accelerated concurrent data structures. `https://github.com/NVIDIA/cuCollections`. GitHub repository. Accessed November 9, 2026.